

DAY 2 & DAY 3 COMPLETE MASTER GUIDE FOR FRIEND B

High-Rate Sensor Ingestion (MPU6050 1 kHz FIFO + I2C DMA) & Fixed-Point Radix-2 FFT DSP Pipeline

PART 1: DAY 2 — MPU6050 1 kHz HARDWARE FIFO & I2C DMA STREAMING

1. The Engineering Challenge: Why Polling Fails at 1,000 Samples/Second

At a 1 kHz sample rate, a new sensor reading arrives every **1.0 millisecond** (1,000 microseconds). If firmware uses standard polling (reading I2C registers in a loop), the CPU wastes >90% of its clock cycles waiting for slow I2C bus transactions. Any momentary CPU delay (e.g., servicing an interrupt or running Wi-Fi/MQTT) causes dropped samples and corrupts the vibration waveform.

2. The Production Solution: Hardware FIFO + Interrupt-Driven DMA Burst

The MPU6050 contains an internal **1024-byte Hardware FIFO** buffer. We configure it to collect accelerometer samples autonomously. When the FIFO has new data, the MPU6050 pulses its physical **INT pin**. The microcontroller's Interrupt Service Routine (ISR) fires and immediately triggers a burst DMA transfer into our **Circular Ring Buffer (from Day 1)** in RAM without any CPU intervention.

MPU6050 Register	Address	Config Value	Engineering Purpose
SMPLRT_DIV	0x19	0x00 (0)	Sets Sample Rate = 1000 Hz / (1 + 0) = 1,000 Samples/sec (1 kHz)
CONFIG	0x1A	0x01	Configures DLPF (Digital Low Pass Filter) to 188 Hz bandwidth
ACCEL_CONFIG	0x1C	0x00	+/- 2g full-scale range (16,384 LSB/g sensitivity)
FIFO_EN	0x23	0x08	Enables ACCEL_FIFO_EN to route Accel X, Y, Z into the 1KB FIFO
INT_ENABLE	0x38	0x01	Enables DATA_RDY_INT to pulse the external INT pin on every sample
USER_CTRL	0x6A	0x44	Bit 6 = FIFO_EN (enable), Bit 2 = FIFO_RESET (flush buffer)

3. Day 2 Mathematical Proof: 400 kHz Fast-Mode I2C Bus Loading Calculation

Let us calculate the exact mathematical bus loading of 1 kHz streaming over a 400 kHz I2C bus:

- Each accelerometer reading = 6 bytes (X_H, X_L, Y_H, Y_L, Z_H, Z_L).
- Protocol overhead per transaction: 1 Start (1b) + Address+W (9b) + RegAddr (9b) + ReStart (1b) + Address+R (9b) + 6 Data Bytes with ACKs (54b) + 1 Stop (1b) = **76 bits total**.
- At 1,000 samples/sec: Total Bandwidth Required = 76 bits x 1,000 Hz = **76,000 bits/sec**.
- At I2C Fast-Mode clock speed of 400,000 bits/sec:
Bus Utilization = (76,000 / 400,000) x 100% = 19.00%.
- **Conclusion:** The I2C bus operates at only 19% capacity, leaving **81% of bus bandwidth completely free**. Zero packets will be dropped even during intense compute bursts!

4. Day 2 Verification Code: firmware/mpu6050_fifo_sim.c

Compile and execute the 60-second real-time stress test (60,000 continuous samples at 1 kHz):

```
gcc firmware/mpu6050_fifo_sim.c -o sim/mpu6050_test.exe && ./sim/mpu6050_test.exe
Output: Total Samples: 60,000 | 128-Sample Windows Prepared: 468 | Dropped Samples: 0 | STATUS: PASS
```

PART 2: DAY 3 — FIXED-POINT RADIX-2 FFT & 16-INT8 FEATURE EXTRACTION

1. Why Time-Domain Vibration Cannot Be Ingested Directly by Neural Networks

In raw vibration signals from industrial machinery (motors, pumps, gearboxes), time-domain samples are noisy, fluctuate rapidly, and are phase-dependent. A bearing defect produces characteristic shockwaves at specific mechanical fault frequencies:

- **BPFO (Ball Pass Frequency Outer Race):** Typically 3.5x to 4x of shaft rotational speed (e.g. 120 Hz).
- **BPFI (Ball Pass Frequency Inner Race):** Typically 5x to 6x of shaft rotational speed (e.g. 180 Hz).

By applying the **Fast Fourier Transform (FFT)** on 128-sample windows, we convert the vibration into the **Frequency Domain**, turning invisible time-domain vibrations into prominent, recognizable energy peaks!

2. Integer FFT: Why We Eliminate Floating-Point Math on Microcontrollers

Standard FFT libraries use 32-bit floats and trigonometry functions (sin, cos) that take hundreds of clock cycles. In `firmware/fixed_fft_dsp.c`, we implement a **Fixed-Point Radix-2 Decimation-In-Time (DIT) Cooley-Tukey FFT**:

- **Twiddle Factor Look-Up Table:** $W_{128}^k = \cos(2\pi k/128) - j\sin(2\pi k/128)$ precomputed in **Q1.14 fixed-point** (scaled by 16,384). Zero runtime trigonometry!
- **Bit-Reversal Permutation:** Swaps array inputs in $O(N)$ using a 7-bit reversal function.
- **Radix-2 Butterfly Arithmetic:** 7 stages ($2^7 = 128$ points). Each butterfly performs: $(vr_{wr} - vi_{wi}) \gg 14$ and scales intermediate values by $\gg 1$ to prevent 16-bit register overflow.

3. Spectral Energy Pooling & Quantization to 16-Element Q1.7 Vector

The 128-point FFT produces 64 positive frequency bins (at 1 kHz Fs, frequency resolution = $1000/128 = 7.8125$ Hz per bin). To feed Friend A's **4x4 INT8 Systolic Array** (which takes 16 inputs simultaneously), we pool every 4 consecutive bins into 16 discrete spectral energy bands:

Band_Energy[i] = Sum of $|X[k]|^2$ for k in $[1 + i*4$ to $1 + (i+1)*4$].

We normalize the peak band to +127 and quantize each band into a **Signed Q1.7 INT8 byte (-128 to +127)**. This 16-byte vector directly enters the neural network accelerator!

Feature Band	FFT Bins	Frequency Range (Hz)	Physical Machine Fault Target
Feature #00	Bins 1 - 4	7.8 - 39.1 Hz	Sub-synchronous vibration, oil whirl, looseness
Feature #01	Bins 5 - 8	39.1 - 70.3 Hz	1X Shaft Rotational Speed (Unbalance, bent shaft)
Feature #02	Bins 9 - 12	70.3 - 101.6 Hz	2X Rotational Speed (Shaft misalignment)
Feature #03	Bins 13 - 16	101.6 - 132.8 Hz	Ball Pass Frequency Outer Race (BPFO = 120 Hz Bearing Fault!)
Feature #04 - #10	Bins 17 - 44	132.8 - 351.6 Hz	Gear meshing frequencies & bearing harmonics
Feature #11 - #15	Bins 45 - 64	351.6 - 507.8 Hz	High-frequency structural resonance & cavitation

4. Day 3 Verification Code: `firmware/fixed_fft_dsp.c` & Dataset Generator

Compile and execute the integer FFT and the 200-sample industrial dataset generator:

```
gcc firmware/fixed_fft_dsp.c -o sim/fft_test.exe && ./sim/fft_test.exe
Output: Peak captured at Feature #03 (101.6 - 132.8 Hz) with 0x7F (+127)! Zero floating-point operations!
python scripts/generate_vibration_dataset.py
Output: Generated 200 feature frames -> sim/input_features.hex for Friend A's systolic array testbench!
```

PART 3: ANNOTATED CORE CODE & DAILY TERMINAL WORKFLOW

Below is the core Radix-2 butterfly loop from `firmware/fixed_fft_dsp.c`. Notice how every multiplication is performed using 16-bit integer math and scaled with bitwise shifts ($\gg 14$ and $\gg 1$):

```
// Core Butterfly Execution in firmware/fixed_fft_dsp.c (Stages 1 through 7):
for (int s = 1; s <= 7; s++) {
    int m = 1 << s;          // Transform size (2, 4, 8, ... 128)
    int m2 = m >> 1;         // Half-size
    int step = 128 / m;      // Twiddle step

    for (int k = 0; k < 128; k += m) {
        for (int j = 0; j < m2; j++) {
            int tw_idx = j * step;
            int16_t wr = twiddle_cos[tw_idx]; // Q1.14 Cosine
            int16_t wi = twiddle_sin[tw_idx]; // Q1.14 Sine

            cplx16_t u = x[k + j];
            cplx16_t v = x[k + j + m2];

            // Complex Multiply in Q1.14: (wr + j*wi) * (v.real + j*v.imag)
            int32_t vr_wr = ((int32_t)v.real * wr) >> 14;
            int32_t vi_wi = ((int32_t)v.imag * wi) >> 14;
            int32_t vr_wi = ((int32_t)v.real * wi) >> 14;
            int32_t vi_wr = ((int32_t)v.imag * wr) >> 14;

            int16_t t_real = (int16_t)(vr_wr - vi_wi);
            int16_t t_imag = (int16_t)(vr_wi + vi_wr);

            // Butterfly Addition and Subtraction (>> 1 prevents overflow!)
            x[k + j].real    = (u.real + t_real) >> 1;
            x[k + j].imag    = (u.imag + t_imag) >> 1;
            x[k + j + m2].real = (u.real - t_real) >> 1;
            x[k + j + m2].imag = (u.imag - t_imag) >> 1;
        }
    }
}
```

Complete Day 2 & Day 3 Terminal Commands for Friend B

1. Update your local repository with latest files from GitHub:
`git pull origin main`

2. Run Day 2 60-Second Sensor FIFO & DMA Ring Buffer Benchmark:
`gcc firmware/mpu6050_fifo_sim.c -o sim/mpu6050_test.exe`
`./sim/mpu6050_test.exe`
Verify: 60,000 samples received, 0 dropped samples, 19% I2C bus load

3. Run Day 3 Fixed-Point 128-Point FFT DSP Pipeline:
`gcc firmware/fixed_fft_dsp.c -o sim/fft_test.exe`
`./sim/fft_test.exe`
Verify: Feature #03 captures 120 Hz peak at +127 (0x7F)

4. Generate 200 Industrial Bearing Feature Vectors for Friend A's Verilog:
`python scripts/generate_vibration_dataset.py`
Verify: `sim/input_features.hex` created with 200 lines

5. Commit and push Day 2 & Day 3 progress to GitHub:
`git add firmware/ scripts/ sim/input_features.hex`
`git commit -m "Day 2 & 3 [Friend B]: Implemented MPU6050 FIFO driver, fixed-point FFT, and bearing dataset"`
`git push origin main`

Day 2 & Day 3 Self-Assessment Quiz for Friend B

Q1: Why does reading MPU6050 samples via hardware FIFO + DMA consume only 19% of a 400 kHz I2C bus? (Answer: Because 1 sample = 76 bits with protocol overhead; $76 \times 1000 \text{ Hz} = 76,000 \text{ bps}$; $76k / 400k = 19\%$.)

Q2: In our 128-point FFT sampled at 1 kHz, what is the exact frequency resolution of each bin? (Answer: $F_s / N = 1000 \text{ Hz} / 128 = 7.8125 \text{ Hz per bin}$.)

Q3: Which feature band captures a 120 Hz outer-race bearing fault? (Answer: Feature #03, spanning bins 13 to 16, which covers 101.6 Hz to 132.8 Hz.)